Task 02 – Packet Sniffing, Firewall Configuration and Vulnerability Scanning

CyArt SOC Internship – Week 01

Intern Name: Harvinder Singh

Date: 19 March 2026

1. Introduction

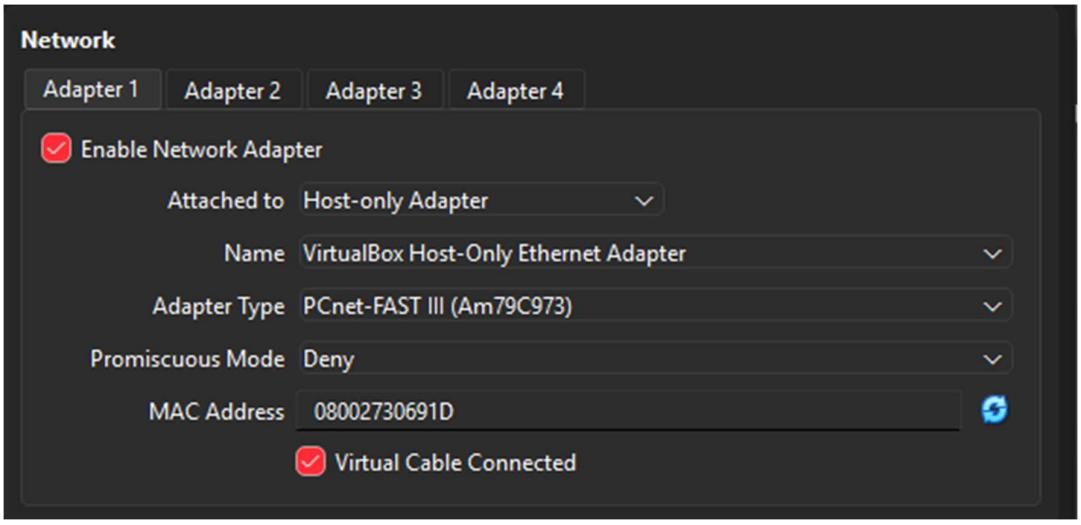
This report explains the work I completed for Task 02 during my SOC internship at CyArt. The goal of this task was to practise packet sniffing using Scapy, configure a basic firewall using iptables, and prepare a lab environment for future vulnerability scanning with OpenVAS.
To do this, I continued using the same virtual lab from Task 01 where Kali Linux acts as the attacker machine and Metasploitable 2 acts as the intentionally vulnerable target. First, I wrote a Python script with Scapy to capture and analyse live network packets. Next, I configured iptables rules on a Linux system to allow only SSH and HTTP while blocking other incoming traffic, and tested the behaviour from another host. Finally, I planned the OpenVAS setup which will later be used to run automated vulnerability scans against the lab target.

2. Lab Environment and Target Details

For this task I used the following setup:

- Host Machine: Windows with Oracle VirtualBox installed
- Attacker VM: Kali Linux
- Target VM: Metasploitable 2
- Virtual Network: Host-only / lab network inside VirtualBox
- Subnet: 192.168.56.0/24
- Metasploitable IP Address: 192.168.56.102

Kali and Metasploitable were both connected to the same virtual network. This created an isolated lab where I could safely capture packets, change firewall rules and prepare for vulnerability scans without affecting any production systems. Kali was used to run Scapy, manage iptables and later act as the OpenVAS server, while Metasploitable 2 remained the deliberately vulnerable target.



3. Packet Sniffing with Scapy

3.1 Scapy Installation and Setup

For packet sniffing I used Scapy, a Python-based packet manipulation tool. Scapy was installed on Kali Linux, and I referred to the official documentation to understand how to use the sniff() function, select an interface, and inspect protocol layers such as TCP, UDP and ICMP.

3.2 Python Script for Packet Capture (packet_sniffer.py)

I created a Python script called packet_sniffer.py that automates a small capture:

- It listens on a selected network interface.
- Captures exactly 100 packets.
- Checks each packet to see whether it contains TCP, UDP or ICMP.
- Increments counters for each protocol type and prints a summary at the end.

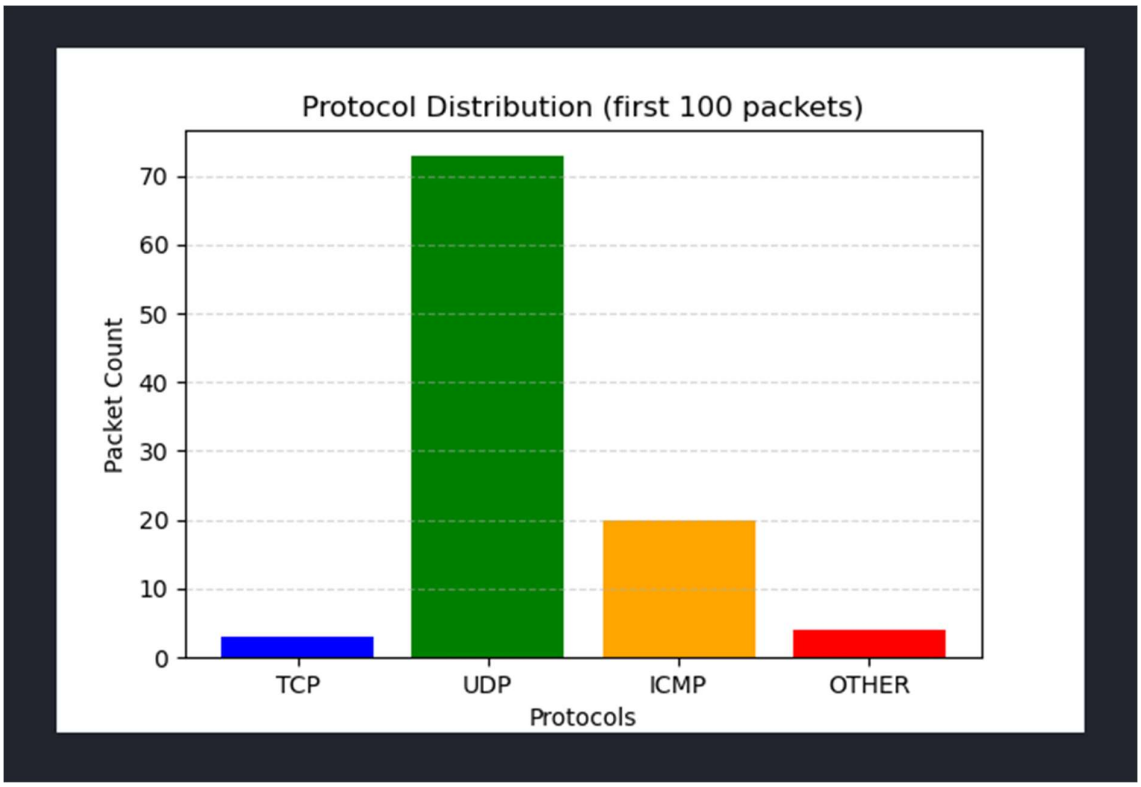
This script allowed me to quickly collect a snapshot of live traffic instead of analysing packets one by one.

```python
from scapy.all import sniff, TCP, UDP, ICMP
import matplotlib.pyplot as plt

def main():
    interface = input("Enter interface name (e.g., eth1): ")

    print(f"[+] Sniffing on interface: {interface}")
    packets = sniff(iface=interface, count=100)

    print(f"[+] Captured packets: {len(packets)}")

    protocol_counts = {"TCP": 0, "UDP": 0, "ICMP": 0, "OTHER": 0}

    for pkt in packets:
        if pkt.haslayer(TCP):
            protocol_counts["TCP"] += 1
        elif pkt.haslayer(UDP):
            protocol_counts["UDP"] += 1
        elif pkt.haslayer(ICMP):
            protocol_counts["ICMP"] += 1
        else:
            protocol_counts["OTHER"] += 1

    print("[+] Protocol distribution:")
    for proto, count in protocol_counts.items():
        print(f"{proto}: {count}")

    # Yaha se chart ka code start
    protocols = list(protocol_counts.keys())
    counts = list(protocol_counts.values())

    plt.figure(figsize=(6, 4))
    plt.bar(protocols, counts, color=['blue', 'green', 'orange', 'red'])
    plt.xlabel("Protocols")
    plt.ylabel("Packet Count")
    plt.title("Protocol Distribution (first 100 packets)")
    plt.grid(axis="y", linestyle="--", alpha=0.5)

    plt.savefig("protocol_distribution.png")
    print("[+] Saved bar chart as protocol_distribution.png")

if __name__ == "__main__":
    main()
```

3.3 Results: Protocol Distribution

I ran the script while generating traffic in the lab (pings to the target, SSH logins and basic web browsing). After capturing 100 packets, the script reported separate counts for TCP, UDP, ICMP and other protocols. Using these counts, I plotted a bar chart that shows how many packets belonged to each protocol type.

The chart clearly showed that TCP packets dominated due to SSH and HTTP, with a smaller number of ICMP packets from ping and some UDP packets depending on background services. This helped me connect the raw packet view from Scapy with a high-level understanding of which protocols are actually active on the network.



4. Basic Firewall Configuration with iptables

4.1 Firewall Objective

The firewall objective in this task was simple:

- Allow SSH (TCP port 22)
- Allow HTTP (TCP port 80)
- Block all other incoming connections

This simulates a hardened server where only management and web traffic are exposed to the network.

4.2 iptables Commands Used

On the Linux machine, I first cleared existing rules and set restrictive default policies, then added explicit allow rules. The main commands were:

```
sudo iptables -F
sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT

sudo iptables -A INPUT -m conntrack --ctstate
ESTABLISHED,RELATED -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
```

These rules make sure that new incoming connections are accepted only on ports 22 and 80, while replies to existing connections are still allowed. All other incoming traffic is dropped by default.

```
└─$ sudo iptables -L -n -v

Chain INPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 ACCEPT     all  --  lo     *       0.0.0.0/0            0.0.0.0/0
    0     0 ACCEPT     all  --  *      *       0.0.0.0/0            0.0.0.0/0            state RELATED,ESTABLISHED
    0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:22
    0     0 ACCEPT     tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:80

Chain FORWARD (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
```

4.3 Firewall Testing and Observations

To confirm that the firewall was working, I performed several tests from another host in the lab:

- Connected to the machine over SSH on port 22 – the connection succeeded.
- Accessed a simple HTTP service on port 80 – the page loaded successfully.
- Ran an Nmap scan against a range of ports – only ports 22 and 80 appeared as open, and other ports showed as closed or filtered.

These results showed that the iptables configuration matched the intended policy: SSH and HTTP were reachable, while other services were effectively blocked.

```
msfadmin@metasploitable:~$ nmap -sS 192.168.56.103

Starting Nmap 4.53 ( http://insecure.org ) at 2026-03-18 09:06 EDT
Stats: 0:00:17 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 30.86% done; ETC: 09:06 (0:00:10 remaining)
Stats: 0:00:18 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 32.67% done; ETC: 09:06 (0:00:10 remaining)
Stats: 0:00:18 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 32.73% done; ETC: 09:06 (0:00:10 remaining)
Stats: 0:00:18 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 33.58% done; ETC: 09:06 (0:00:10 remaining)
Interesting ports on 192.168.56.103:
Not shown: 1712 filtered ports
PORT   STATE  SERVICE
22/tcp closed ssh
80/tcp open   http
MAC Address: 08:00:27:88:C5:F0 (Cadmus Computer Systems)

Nmap done: 1 IP address (1 host up) scanned in 21.789 seconds
```

5. Vulnerability Scanning with OpenVAS (In Progress)

As part of Task 02 I also started working on setting up Greenbone Vulnerability Management (OpenVAS) on the Kali machine so that I can run an automated vulnerability scan against the Metasploitable VM (192.168.56.102). I installed the GVM package, went through the initial setup steps and was able to reach the web interface.
However, the default scan configurations were not created correctly in this environment, so no Scan Config options appear in the GUI and new scan tasks cannot be started yet. Because of this GVM issue, I treated the OpenVAS work as in progress for this week and focused the main report on the packet sniffing and firewall tasks. Once the scan configuration problem is resolved, I will run a full OpenVAS scan and add a short follow-up report with the detailed vulnerability findings.

6. Key Learnings

- Writing a custom packet sniffer with Scapy helped me see real TCP, UDP and ICMP traffic in my own lab instead of only reading theory.
- Summarising the captured packets in a bar chart made it much easier to understand which protocols were actually active during the test.
- A small number of carefully chosen iptables rules is enough to harden a Linux system by exposing only required services such as SSH and HTTP.

- Testing the firewall with Nmap and simple connection attempts is important to confirm that the rules work as expected and that no extra ports are open.
- The OpenVAS part showed that larger security tools have multiple moving pieces (services, databases, feeds), and that documenting environment limitations is also part of real SOC work.